

This is a copy of a shared ChatGPT conversation
[Report conversation](#)

Make an app that people can buy different type of public transportation tickets such as single ticket, period ticket, or a bundle of tickets. Users can buy tickets from a mobile app or any digital interface such as websites. Any user can buy more than one ticket att the same time and it's important that the app handle it. the will be discount for students and seniors while adaults pay in full price. Write the code in python

I'm building a small but working Python ticket app, not just a sketch. It will support single, period, and bundle tickets, full-price and discounted users, and buying multiple tickets in one order.

Thought for 17s >

Done — here is the Python app file:

[Download ticket_app.py](#)

What it does:

- sells **single**, **period**, and **bundle** tickets
- supports **adult**, **student**, and **senior** prices
- lets one user buy **multiple tickets in the same order**
- stores **order history**
- exposes a simple **Flask API** for mobile app or website use

How to run it:

Bash

```
pip install flask
python ticket_app.py
```

Then open:

Bash

http://127.0.0.1:5000

Main endpoints:

- GET /catalog
- POST /users
- POST /orders
- GET /users/<user_id>/orders

Example user creation:

JSON

```
{
  "name": "Parham",
  "passenger_type": "student"
}
```

Example order:

JSON

```
{
  "user_id": "PUT_USER_ID_HERE",
  "items": [
    {"ticket_type": "single", "quantity": 2},
    {"ticket_type": "bundle", "quantity": 1},
    {"ticket_type": "period", "quantity": 1}
  ]
}
```

I can also turn this into a version with:

- a real database
- login system
- payment simulation
- simple frontend webpage </>

I see the whole code is in a comment. I want a working python code. Make sure that no matter the pressure of users and on the server, the app will work.

You were right to call that out. I'm replacing it with a real runnable app file and making the order flow safer under concurrent requests by using database transactions, validation, and

thread-safe handling.

Thought for 10s >

I replaced it with a real runnable Python file:

[Download transport_ticket_app.py](#)

Run it like this:

Bash

```
pip install flask
python transport_ticket_app.py
```

Then open:

Bash

```
http://127.0.0.1:5000
```

Important:

- This version is much safer under concurrent use because it uses:
 - SQLite persistence
 - atomic database transactions
 - write locking
 - input validation
 - structured error handling
- No app can honestly guarantee it will work under **any** pressure forever. That depends on server size, database choice, load balancer, deployment, and monitoring.
- But this code is a solid working base and is much better for multi-user pressure than a simple in-memory script.

It supports:

- single tickets
- period tickets
- bundle tickets
- adult, student, senior pricing
- multiple tickets in one order
- saved users
- saved order history

Main endpoints:

- GET /catalog

- POST /users
- GET /users/<user_id>
- POST /orders
- GET /users/<user_id>/orders
- GET /health

For real production, use:

Bash

```
gunicorn -w 4 -b 0.0.0.0:5000 transport_ticket_app:app
```

If you want, I can next make:

1. a real frontend webpage for this app, or
2. a stronger production version with login, JWT, Docker, and PostgreSQL. </>